

INFO-F307

Bugémon

Groupe: 10



PLAN DE LA PRÉSENTATION

01 Génie logiciel

02 Gestion de projet



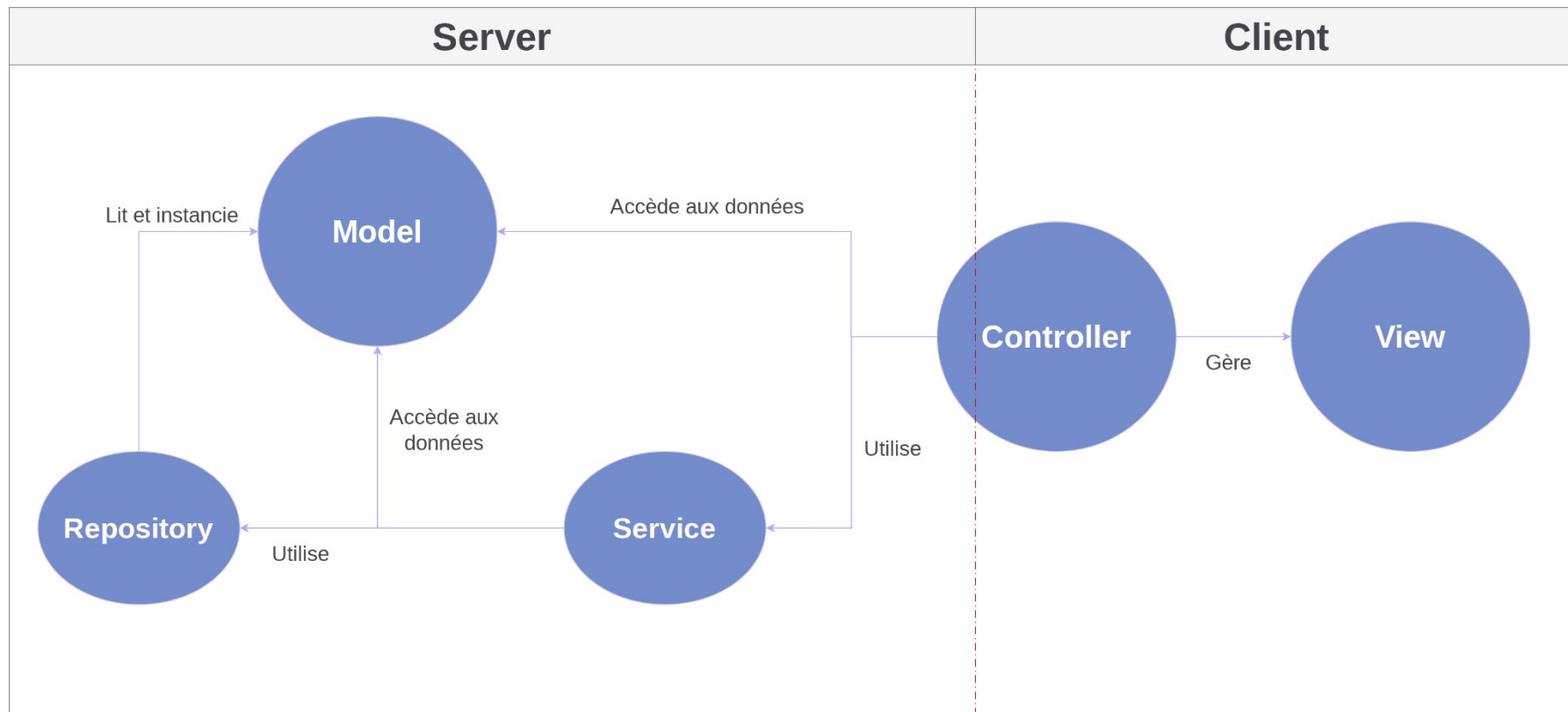
01

Génie logiciel

ARCHITECTURE MVC AVANCÉE

4

Avec services et dépôts



SERVICES & DÉPÔTS

5

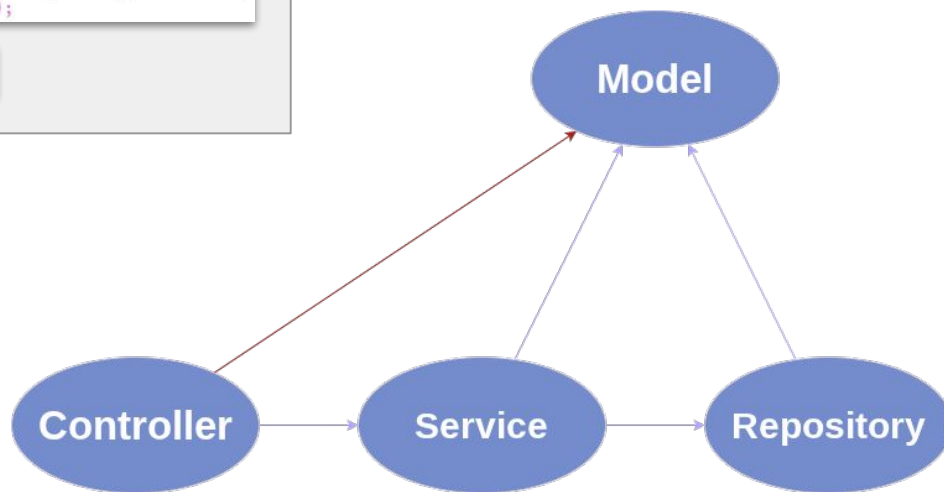
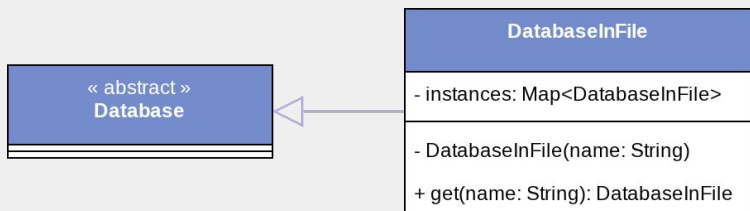
Ajoutés progressivement

Injection de dépendance

```
BugemonSpeciesRepository bugemonSpeciesRepository = new BugemonSpeciesDatabaseRepository(database);  
BugemonService bugemonService = new BugemonService(bugemonSpeciesRepository);
```

```
BugemonSpeciesRepository repository = new BugemonSpeciesMockRepository();  
BugemonService service = new BugemonService(repository);
```

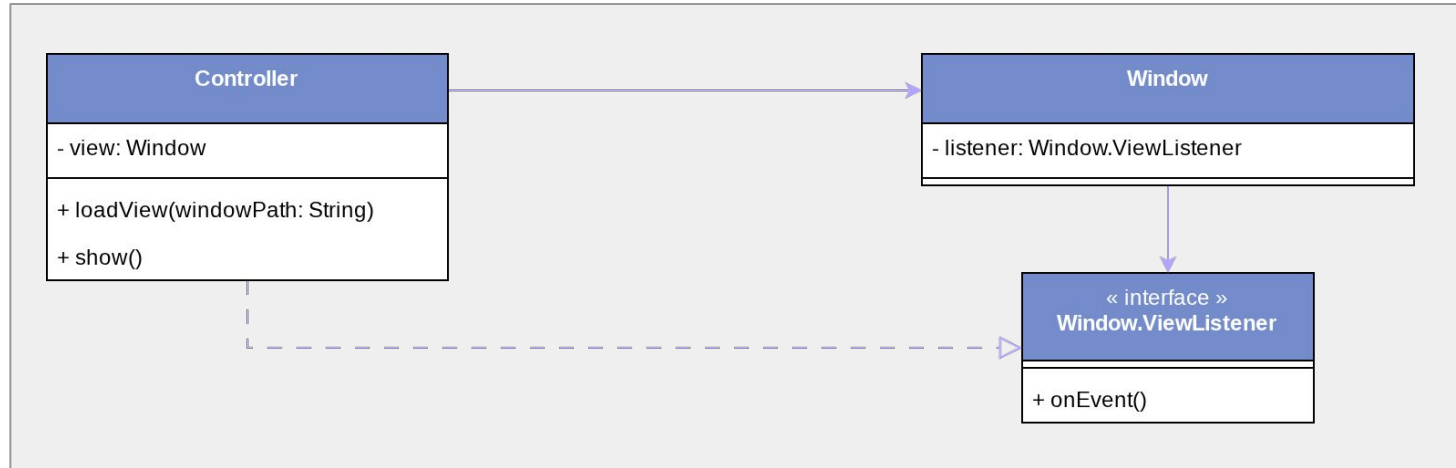
Singleton (multiton)



VUE - CONTRÔLEUR

6

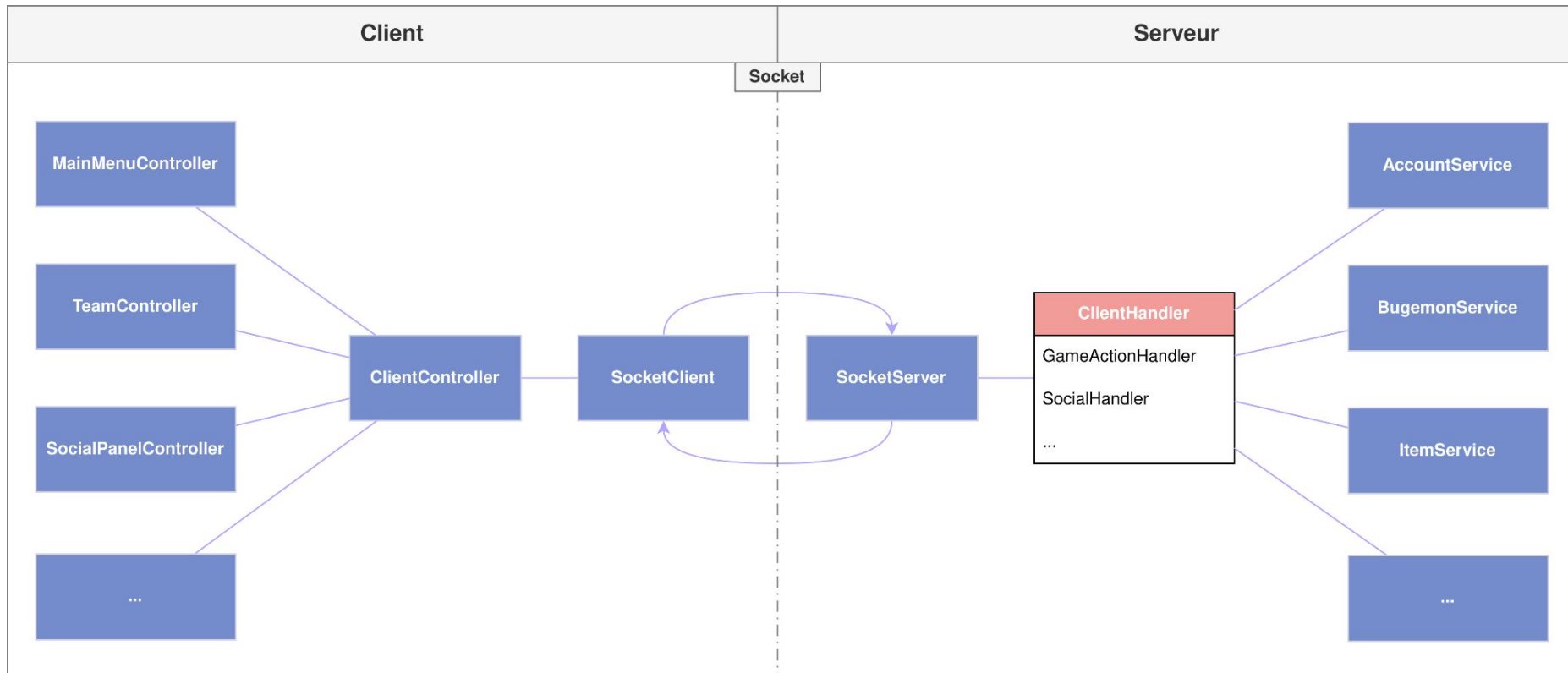
Listener



CLIENT - SERVEUR

7

Médiateur et Façade



CLIENT - SERVEUR

8

Simple distribution

Avant

```
if (message instanceof UseAbilityRequest r) {
    >> chooseAbilityAction(r.getAbility());
}
else if (message instanceof RegisterRequest r) {
    >> registerPlayer(r.getDTO(), r.isLogin());
}
else if (message instanceof AbandonTowerRequest r) {
    >> abandonTower();
}
else if (message instanceof ChooseItemRewardRequest r) {
    >> chooseItemReward(item);
}
else if (message instanceof ChooseLevelUpRewardRequest r) {
    >> chooseLevelUpReward(reward);
}
else if (message instanceof ChooseRandomActionRequest r) {
    >> chooseRandomAction();
}
else if (message instanceof ChooseStatRewardRequest r) {
    >> chooseStatReward(bugemon);
}
else if (message instanceof ChooseTowerRoomRequest r) {
    >> chooseTowerRoom(roomId);
}
else if (message instanceof QuitMultiBattleRequest r) {
    >> quitMultiBattle(opponent);
}
else if (message instanceof RunRequest) {
    >> chooseRunAction();
}
// ...
```

Après

```
public class ClientHandler extends Thread implements ServerMessageHandler {
    >> private void handleMessage() {
    >>     Request message = this.receiveMessage();
    >>     message.dispatch(this);
    >> }
    >> @Override
    >> public void saveTeam(TeamDTO teamDTO) { teamSaveHandler.saveTeam(teamDTO); }
}
```

```
public class SaveTeamRequest implements Request {
    >> @Override
    >> public void dispatch(ServerMessageHandler handler) {
    >>     handler.saveTeam(this.team);
    >> }
}
```

1

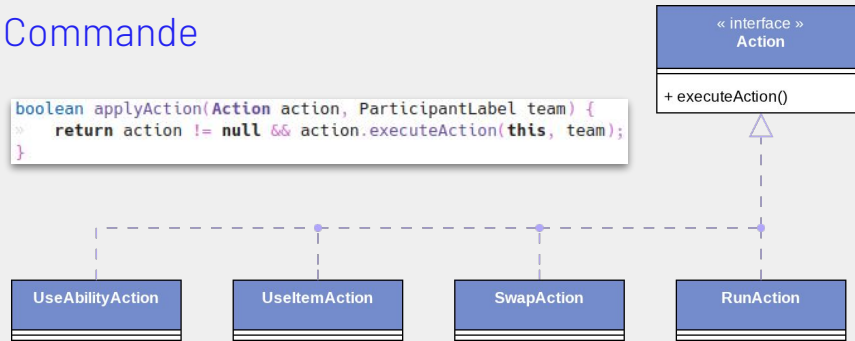
2

AUTRES PATRONS

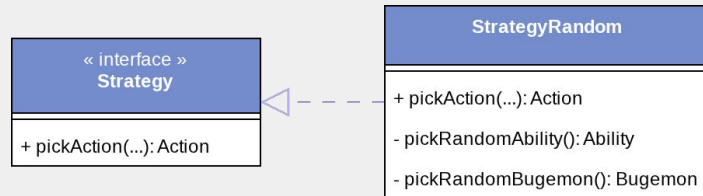
9

Commande

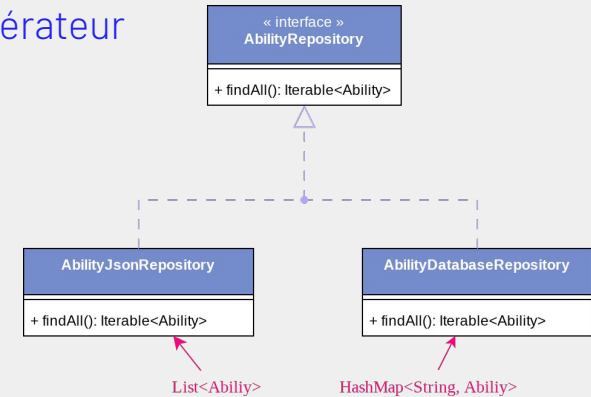
```
boolean applyAction(Action action, ParticipantLabel team) {  
    return action != null && action.executeAction(this, team);  
}
```



Stratégie



Itérateur





NOTRE MÉTHODE DE PILOTAGE

Nous avons progressivement transformé un suivi “liste de tâches” en un vrai outil de coordination.

Suivi centralisé

Google Doc avec statuts, estimation, temps réel et responsable de chaque tâche (lors de chaque séance de pair programming).

Pair programming

- pendant une semaine, on gardait le même binôme
- essai de binôme “sauvage” → pas marché

Organisation des tâches 4

Légende

Tâches à faire (+ prioritaire)

Tâches en cours

Tâches accomplies

Changements (refactoring)

Tâches accomplies (semaine bonus)

Mettre près de chaque tâche qui était (co-)pilote et le temps mis + additionner dans temps mis

Histoires

Sauvegarder et charger une équipe (2)

Tâches

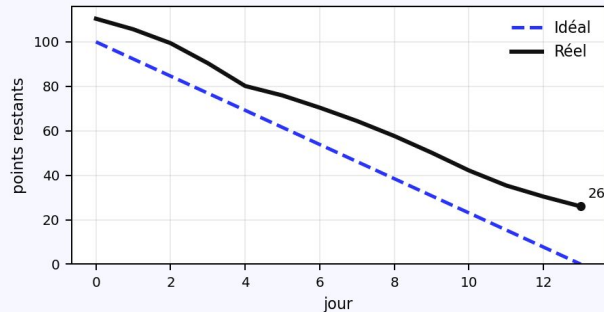
- Sauvegarder une équipe dans la bdd - Rugile (pilote) et Alexis – estimé : 2h – mis : 2h - dans branche BugemonTeam
- Retirer BugemonSpeciesId dans la table Team et mettre BugemonId à la place +

(Sous-)Tâches non-compréhensibles

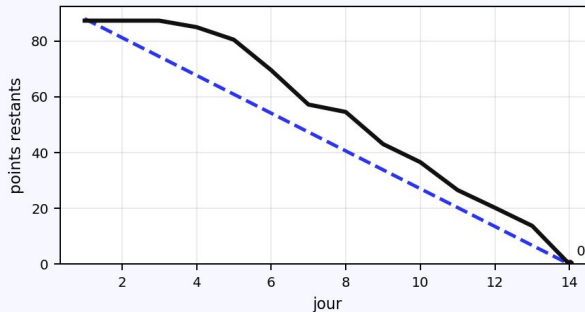
- Tâche normale → binôme l'accomplit
- Tâche difficile → demander de l'aide

BURNDOWN CHARTS : APPRENTISSAGE EN 4 ITÉRATIONS

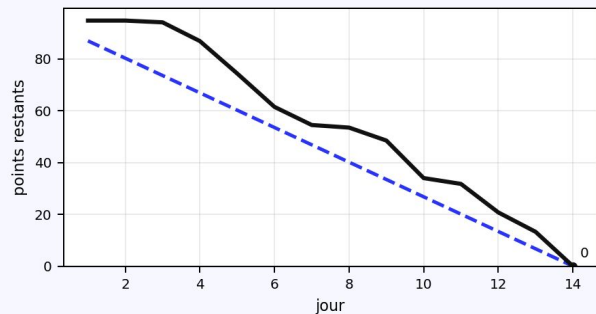
Itération 1



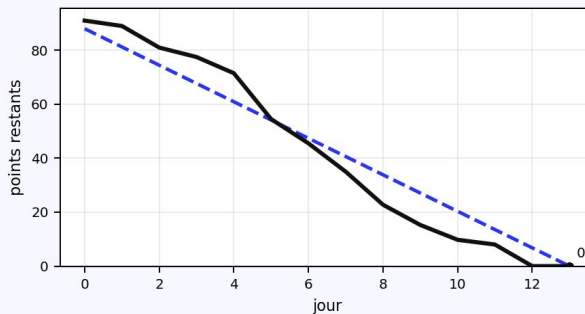
Itération 2



Itération 3



Itération 4



Statistiques

286 TÂCHES

It.1: 69 It.2: 27 It.1: 22 It.2: 19
It.3: 56 It.4: 58 It.3: 18 It.4: 19

263 tests JUnit — 60 classes de test

269 fichiers Java — 17,9k lignes main

16 vues FXML — 21 contrôleurs de vue

17 histoires **COMPLÉTÉES**

18 réunions d'équipe

**MERCI
POUR
VOTRE
ÉCOUTE!**

